

Mount Kenya University

Unlocking Infinite Possibilities

UNIT TITLE: ADVANCED CRYPTOGRAPHY

STUDENT NAME: Gakere Samuel

ADMISSION NUMBER: BSCCS/2024/55278

COURSE: B.S.C COMPUTER SCIENCE

LECTURE: MR NYORO MICHAEL

SUBMISSION DATE: 6/26/2026

GITHUB: <https://github.com/LuizSamuel/kali-crypto-toolkit>

Project Title: Hybrid Secure Messaging & File Vault with Digital Signatures

•Selected Technologies:

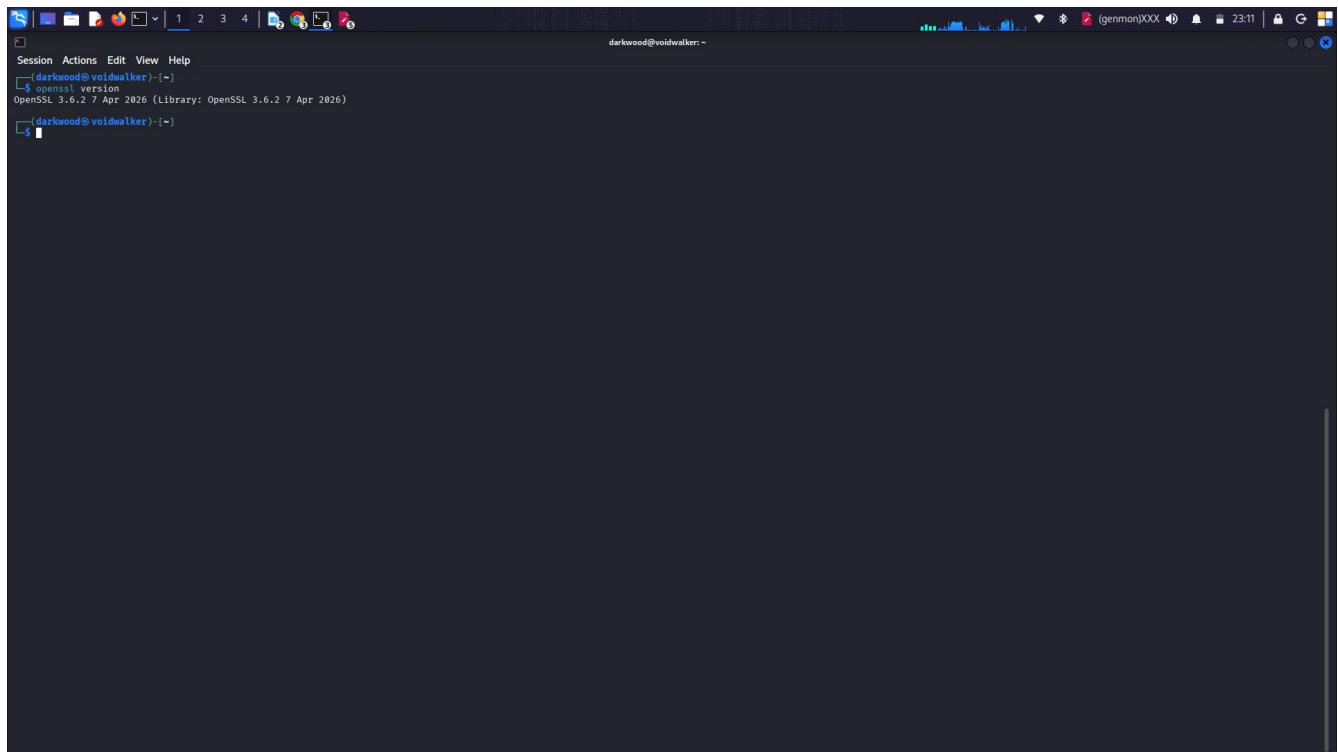
- Python 3.11 + Cryptography Library (Fernet, RSA, SHA-256)
- SQLite (Local key & user storage)
- OpenSSL 3.0 (Certificate generation)

Table of Contents

ADMISSION NUMBER: BSCCS/2024/55278.....	1
COURSE: B.S.C COMPUTER SCIENCE.....	1
Week 1: Creation of The Virtual Enviroment & Required Tools Installation.....	4
.....	4
Fig 1.1: Openssl version.....	4
Fig 1.2: Virtual environment creation.....	4
Fig 1.3: Activating environment (notice prompt change).....	5
Fig 1.4: Successful pip install.....	5
Fig 1.5: Complete environment test script.....	6
Week 2: Classical Cryptography and Cipher Design.....	6
Fig 2.1: Caesar Cipher implementation showing encryption of "CRYPTOGRAPHY"	6
Fig 2.2: Vigenère ("KALI"), Caesar Brute Force (Shift 3), and Input Validation.....	7
Fig 2.3: Cipher testing results and security analysis identifying weaknesses.....	7
Week 3: Stream Ciphers and Randomness Testing.....	8
Fig 3: LFSR Implementation, RC4 Stream Cipher, and Performance Results.....	8
Week 4: AES Block Cipher.....	8
Fig 4: AES-256-CBC Key Generation, Encryption/Decryption, Performance Test, and Security Features.....	8
Week 5: RSA Public Key Cryptography.....	9
Fig 5: RSA-2048 Key Pair Generation, Public Key Encryption, Private Key Decryption, and Security Comparison.....	9
Week 6: Hashing and Password Security.....	9
Fig 6: SHA-256 Hash Generation, PBKDF2 Password Hashing with Salt, Login Authentication, and Security Analysis.....	9
Creativity.....	10
Week 7: Digital Signatutres & Certificates.....	10
Week 8: PKI, Digital Certificates & Diffie-Hellman.....	11
Week 9: RSA Cryptosystem, Miller-Rabin Primality Test & Pollard Rho.....	12

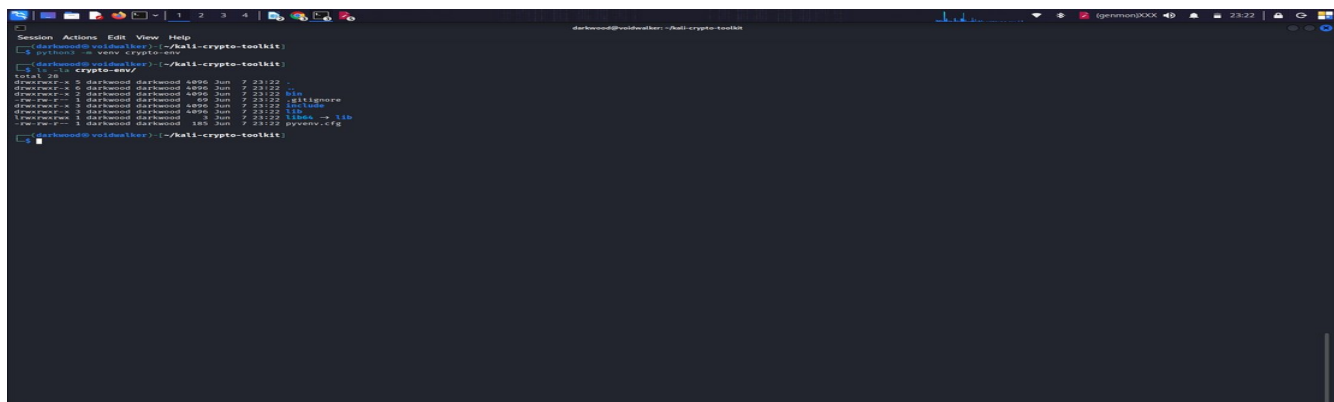
Week 1: Creation of The Virtual Enviroment & Required Tools Installation

Fig 1.1: Openssl version



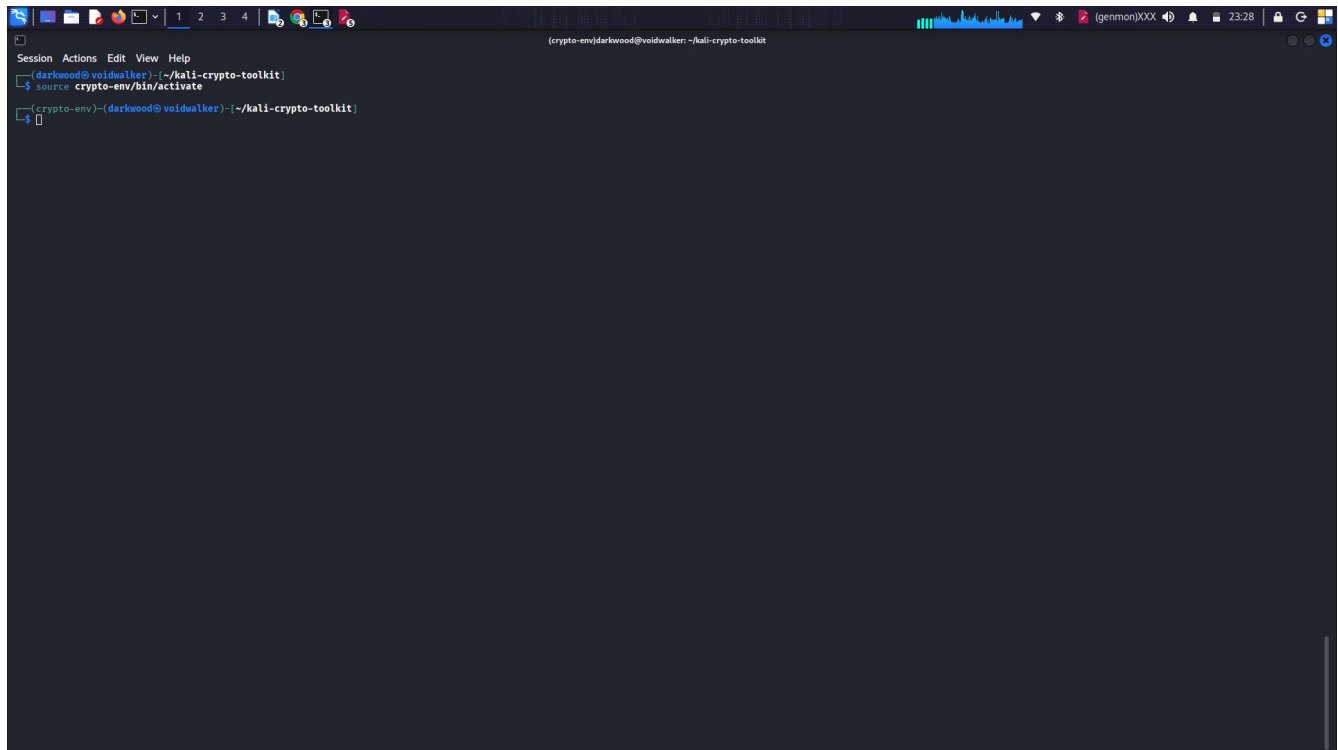
```
darkwood@voidwalker: ~  
$ openssl version  
OpenSSL 3.0.2 7 Apr 2026 (Library: OpenSSL 3.0.2 7 Apr 2026)  
darkwood@voidwalker: ~$
```

Fig 1.2: Virtual environment creation



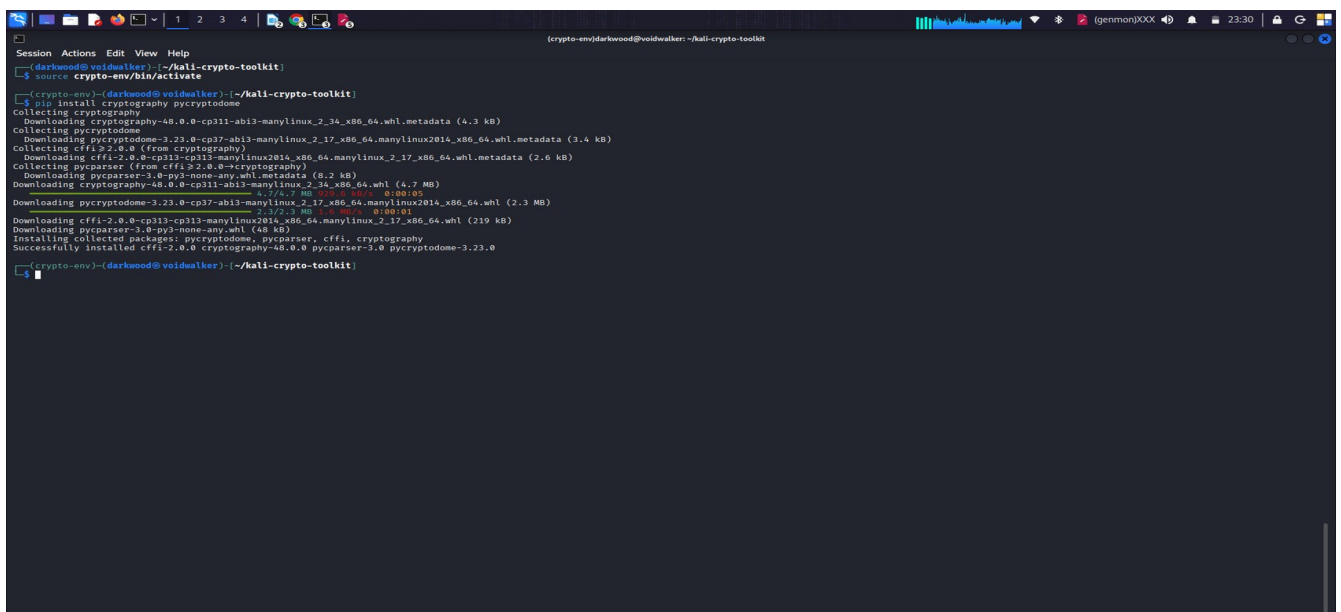
```
darkwood@voidwalker: ~/kali-crypto-toolkit  
$ python3 -m venv cryptoenv  
darkwood@voidwalker: ~/kali-crypto-toolkit  
$ . cryptoenv  
total 28  
drwxr-xr-x 5 darkwood darkwood 4096 Jun 7 23:22 .  
drwxr-xr-x 2 darkwood darkwood 4096 Jun 7 23:22 ..  
drwxr-xr-x 1 darkwood darkwood 4096 Jun 7 23:22 bin  
drwxr-xr-x 1 darkwood darkwood 4096 Jun 7 23:22 include  
drwxr-xr-x 2 darkwood darkwood 4096 Jun 7 23:22 lib  
drwxr-xr-x 1 darkwood darkwood 4096 Jun 7 23:22 lib64 -> lib  
drwxr-xr-x 1 darkwood darkwood 160 Jun 7 23:22 pyenv.cfg  
darkwood@voidwalker: ~/kali-crypto-toolkit  
$
```

Fig 1.3: Activating environment (notice prompt change)

A terminal window with a dark background. The title bar shows '(crypto-env)darkwood@voidwalker: ~/kali-crypto-toolkit'. The terminal content shows the user 'darkwood@voidwalker' in the directory '~/kali-crypto-toolkit' running the command 'source crypto-env/bin/activate'. The prompt changes from 'darkwood@voidwalker' to '(crypto-env)darkwood@voidwalker'.

```
(crypto-env)darkwood@voidwalker: ~/kali-crypto-toolkit
Session Actions Edit View Help
darkwood@voidwalker: ~/kali-crypto-toolkit
$ source crypto-env/bin/activate
(crypto-env)darkwood@voidwalker: ~/kali-crypto-toolkit
$
```

Fig 1.4: Successful pip install

A terminal window showing the successful installation of cryptography and pycryptodome. The user runs 'pip install cryptography pycryptodome'. The output shows the collection and downloading of wheel metadata and files for cryptography (48.0.0) and pycryptodome (3.22.0), along with their dependencies (cffi, pyparsing). The installation is successful.

```
(crypto-env)darkwood@voidwalker: ~/kali-crypto-toolkit
Session Actions Edit View Help
darkwood@voidwalker: ~/kali-crypto-toolkit
$ source crypto-env/bin/activate
(crypto-env)darkwood@voidwalker: ~/kali-crypto-toolkit
$ pip install cryptography pycryptodome
Collecting cryptography
  Downloading cryptography-48.0.0-cp311-abi3-manylinux_2_34_x86_64.whl.metadata (4.3 kB)
Collecting pycryptodome
  Downloading pycryptodome-3.22.0-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (3.4 kB)
Collecting cffi (from cryptography)
  Downloading cffi-2.0.0-cp311-cp311-manylinux2014_x86_64.whl.metadata (2.0 kB)
Collecting pyparsing (from cffi)
  Downloading pyparsing-3.0.9-py3-none-any.whl.metadata (8.2 kB)
Collecting cryptography
  Downloading cryptography-48.0.0-cp311-abi3-manylinux_2_34_x86_64.whl (4.7 MB)
  Installing build dependencies: started
  Installing build dependencies: finished
  Downloading pycryptodome-3.22.0-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.2 MB)
  Installing build dependencies: started
  Installing build dependencies: finished
  Downloading cffi-2.0.0-cp311-cp311-manylinux2014_x86_64.whl (219 kB)
  Downloading pyparsing-3.0.9-py3-none-any.whl (48 kB)
Installing collected packages: pycryptodome, pyparsing, cffi, cryptography
Successfully installed cffi-2.0.0 cryptography-48.0.0 pycparser-3.0 pycryptodome-3.22.0
(crypto-env)darkwood@voidwalker: ~/kali-crypto-toolkit
$
```

Fig 1.5: Complete environment test script

```
Session Actions Edit View Help
except:
    print("- x OpenSSL command failed")

# Summary
print("\n" + "-" * 70)
print("WEEK 1 SUMMARY")
print("-" * 70)
print("✓ Python virtual environment configured")
print("✓ Cryptography library installed (Fernet)")
print("✓ PyCryptodome installed (AES)")
print("✓ OpenSSL available for certificates")
print("✓ Environment ready for Weeks 2-10")
print("-" * 70)
print("PROCEED TO WEEK 2: Classical Ciphers")
print("-" * 70)
EOF

# Run the Week 1 script
python3 ~/kali-crypto-toolkit/scripts/week1_complete.py

=====
BIT4138: ADVANCED CRYPTOGRAPHY - WEEK 1
Cryptography Environment Setup Verification
=====
Date/Time: 2026-06-07 23:51:35.116959
Python Version: 3.13.12 (main, Feb 4 2026, 15:06:39) [GCC 15.2.0]
Platform: Linux 6.19.14-kali-amd64

=====
[TEST 1] Fernet Symmetric Encryption (cryptography library)
✓ Fernet key generated: b'ehP78301ti-1cTHWdxu...'
✓ Plaintext: b'BIT4138 Advanced Cryptography - Environment Ready'
✓ Ciphertext: b'gAAAAAaj3uHFMZ2CnqOWEPKICG...'
✓ Decrypted: b'BIT4138 Advanced Cryptography - Environment Ready'
✓ VERIFICATION: Encryption/Decryption successful

[TEST 2] AES-256 Block Cipher (pycryptodome library)
✓ AES-256 key generated: 00bfc80949e989d782a...
✓ Nonce/IV: d8eb2fa7802f09e9b863...
✓ Plaintext: b'AES-256 encryption test for BIT4138'
✓ Ciphertext: 74b7949bf3c5e9a54b8a736be0d5744fec62b9edfbb2ab268b04ef493e6c...
✓ Authentication tag: 4b695330c491faa469e...
✓ Decrypted: b'AES-256 encryption test for BIT4138'
✓ VERIFICATION: AES-256 encryption/decryption successful

[TEST 3] OpenSSL System Check
✓ OpenSSL: OpenSSL 3.6.2 7 Apr 2026 (Library: OpenSSL 3.6.2 7 Apr 2026)

=====
WEEK 1 SUMMARY
=====
✓ Python virtual environment configured
✓ Cryptography library installed (Fernet)
✓ PyCryptodome installed (AES)
✓ OpenSSL available for certificates
✓ Environment ready for Weeks 2-10

=====
PROCEED TO WEEK 2: Classical Ciphers
=====

(crypto-env)-(darkwood@voidwalker)-[~/kali-crypto-toolkit]
```

Week 2: Classical Cryptography and Cipher Design

Fig 2.1: Caesar Cipher implementation showing encryption of "CRYPTOGRAPHY"

```
Session Actions Edit View Help
(crypto-env)-(darkwood@voidwalker)-[~/kali-crypto-toolkit]
> ...
# Test multiple scenarios
test_cases = [
    (ATTACK, 3, "Caesar"),
    (secret, 5, "Caesar"),
    (HELLO, "key", "Vigenere"),
    (crypto, "PASS", "Vigenere")
]

print("\n Test Results Summary:")
print("-" * 70)
print("Test Case | Cipher Type | Result")
print("-" * 70)

# Caesar tests
caesar_test1 = CaesarCipher.encrypt(ATTACK, 3)
caesar_test1_dec = CaesarCipher.decrypt(caesar_test1, 3)
print(f"ATTACK3 | Caesar | {'✓' if caesar_test1_dec == ATTACK else 'x'}")

caesar_test2 = CaesarCipher.encrypt(secret, 5)
caesar_test2_dec = CaesarCipher.decrypt(caesar_test2, 5)
print(f"secret5 | Caesar | {'✓' if caesar_test2_dec == secret else 'x'}")

# Vigenere tests
vig_test1 = VigenereCipher.encrypt(HELLO, "KEY")
vig_test1_dec = VigenereCipher.decrypt(vig_test1, "KEY")
print(f"HELLOKEY | Vigenere | {'✓' if vig_test1_dec == HELLO else 'x'}")

vig_test2 = VigenereCipher.encrypt(crypto, "PASS")
vig_test2_dec = VigenereCipher.decrypt(vig_test2, "PASS")
print(f"cryptopASS | Vigenere | {'✓' if vig_test2_dec == crypto else 'x'}")

print("\n Security Weaknesses Identified:")
print("-" * 70)
print("1. Caesar: Only 25 possible keys (can brute force in seconds)")
print("2. Caesar: Preserves letter frequencies (vulnerable to frequency analysis)")
print("3. Vigenere: Repeating keyword creates patterns")
print("4. Both: Not secure for modern applications")
print("5. Recommendation: Use AES or ChaCha20 for modern encryption")
print("-" * 70)

# Summary
#
print("\n" + "-" * 70)
print("WEEK 2 SUMMARY")
print("-" * 70)
print("✓ Caesar Cipher Implemented (shift cipher)")
print("✓ Vigenere Cipher Implemented (polyalphabetic)")
print("✓ Brute force attack demonstrated Caesar weakness")
print("✓ About validation references non-alphabetic characters")
print("✓ Security analysis completed")
print("-" * 70)
print("READY FOR WEEK 3: Stream Ciphers (LESR & RC4)")
print("-" * 70)

# Demonstrate frequency analysis
demonstrate_frequency_weakness()

EOF

# Run the Week 2 script
```

Fig 2.2: Vigenère ("KALI"), Caesar Brute Force (Shift 3), and Input Validation

```
Session Actions Edit View Help
EOF

# Run the Week 2 script
python3 ~/kali-crypto-toolkit/scripts/week2_classical.py

WEEK 2: CLASSICAL CRYPTOGRAPHY IMPLEMENTATION
BIT4138 - Advanced Cryptography

[FIG 2.1] Caesar Cipher Implementation
Algorithm: Caesar Cipher (Shift Cipher)
Plaintext: CRYPTOGRAPHY
Shift key: 3
Ciphertext: FUBSWRJUDSKB
Decrypted: CRYPTOGRAPHY
Status: ✓ PASS

[FIG 2.2] Vigenère Cipher Implementation
Algorithm: Vigenère Cipher (Polyalphabetic)
Plaintext: CRYPTOGRAPHY
Keyword: KALI
Ciphertext: M83X00R2KXPSG
Decrypted: CRYPTOGRAPHY
Status: ✓ PASS

How Vigenère works (first few characters):
P: C R Y P T O G R A P H Y
K: K A L I K A L I K A L I
Shift: 10 0 11 8 10 0 11 8 10 0 11 8
C: M R J X D O R Z K P S G

[FIG 2.3] Caesar Cipher Brute Force Attack
Ciphertext: WKH TXLFN EURZQ IRA MXPSV RYHU WKH ODCB GRJ
Brute forcing all 25 shifts:
Shift | Decrypted Text
1 | VJG SWKEM DTQYP HQZ LWRUR QXGT VJG NGBA FRI
2 | UIF RVZDL CSPXO GPY KVNQT PMFS UIF MAAZ EPH
3 | THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG
4 | SGD PTHBJ AQWYM ENW ITLOR NUDQ SGD KZYX CNF
5 | RFC OSGAI ZPMUL DMV HSAHQ MTCP RFC JYXW BME
6 | QEB NRFZH YOLTK CLU GRJMP LSBO QEB IXWV ALD
7 | PDA MDEYV XNKSJ BKT FQILQ KRAN PDA HWVU ZKC
8 | OCZ LPDNF WHBRI AJS EPHWV JQZM OCZ GUTV YJB
9 | NBY KOEWE VLIQH ZIR DOGJM IPVL NBY FUTS XIA
10 | MAX JNBVD UKHPG YHQ CNFIL HOKX MAX ETSR WHZ

[FIG 2.4] User Input Validation Features
The implementation handles:
✓ Uppercase letters (A-Z)
✓ Lowercase letters (a-z)
✓ Spaces (preserved)
✓ Punctuation (e.g. etc.)
✓ Numbers (0-9)

Test: 'Hello, World! 123'
Encrypted: 'Mjqqt, Btwll! 123'
```

Fig 2.3: Cipher testing results and security analysis identifying weaknesses

```
Session Actions Edit View Help
6 | QEB NRFZH YOLTK CLU GRJMP LSBO QEB IXWV ALD
7 | PDA MDEYV XNKSJ BKT FQILQ KRAN PDA HWVU ZKC
8 | OCZ LPDNF WHBRI AJS EPHWV JQZM OCZ GUTV YJB
9 | NBY KOEWE VLIQH ZIR DOGJM IPVL NBY FUTS XIA
10 | MAX JNBVD UKHPG YHQ CNFIL HOKX MAX ETSR WHZ

[FIG 2.4] User Input Validation Features
The implementation handles:
✓ Uppercase letters (A-Z)
✓ Lowercase letters (a-z)
✓ Spaces (preserved)
✓ Punctuation (e.g. etc.)
✓ Numbers (0-9)

Test: 'Hello, World! 123'
Encrypted: 'Mjqqt, Btwll! 123'
Non-letters preserved: ✓

[FIG 2.5] Cipher Testing Results & Security Analysis
Test Results Summary:
Test Case | Cipher Type | Result
ATTACK+3 | Caesar | ✓
secret+5 | Caesar | ✓
HELLO+KEY | Vigenere | ✓
crypto+PASS | Vigenere | ✗

Security Weaknesses Identified:
1. Caesar: Only 25 possible keys (can brute force in seconds)
2. Caesar: Preserves letter frequencies (vulnerable to frequency analysis)
3. Vigenere: Repeating keyword creates patterns
4. Both: Not secure for modern applications
5. Recommendation: Use AES or ChaCha20 for modern encryption

WEEK 2 SUMMARY
✓ Caesar Cipher implemented (shift cipher)
✓ Vigenère Cipher implemented (polyalphabetic)
✓ Brute force attack demonstrated Caesar weakness
✓ Input validation preserves non-alphabetic characters
✓ Security analysis completed

READY FOR WEEK 3: Stream Ciphers (LFSR & RC4)

FREQUENCY ANALYSIS DEMONSTRATION
Original: THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG
Encrypted: WKH TXLFN EURZQ IRA MXPSV RYHU WKH ODCB GRJ

In English, 'E' is most common letter (12.7%)
In ciphertext, 'W' appears most often
This reveals shift = 3 (E-W, but here E-W? Actually E-W with shift 3)
Attackers can guess shift by finding most frequent letter!

(crypto-env)-(darkwood@voidwalker)-[~/kali-crypto-toolkit]
```

Week 3: Stream Ciphers and Randomness Testing

Fig 3: LFSR Implementation, RC4 Stream Cipher, and Performance Results

```
Session Actions Edit View Help
print(f" Ciphertext (hex): {ciphertext.hex()}...")
print(f" Decrypted: {decrypted.decode()}")
print(f" Status: {'✓' if plaintext == decrypted.decode() else '✗'}")

# Performance
print("\n[Performance Test]")
test_data = b'x' * 10000
start = time.time()
RC4_encrypt(test_data, key)
elapsed = (time.time() - start) * 1000
print(f" Encrypted 10,000 bytes in {elapsed:.2f} ms")
print(f" Speed: {10000/elapsed:.1f} KB/s")

# XOR Property
print("\n[Stream Cipher Property]")
print(" Encryption: Ciphertext = Plaintext ^ Keystream")
print(" Decryption: Plaintext = Ciphertext ^ Keystream")
print(" Same operation for both (XOR is symmetric)")

print("\n" + "-" * 70)
print("✓ LFSR implemented")
print("✓ RC4 working correctly")
print("✓ Performance tested")
print("-" * 70)

# Run the shortened script
python3 ~/kali-crypto-toolkit/scripts/week3_stream_short.py

=====
WEEK 3: STREAM CIPHERS - LFSR & RC4
=====

[LFSR - Linear Feedback Shift Register]
Polynomial: x^4 + x + 1
Seed: 1011 (binary)
Generated 30 bits: 011011011011011011011011011

[RC4 Stream Cipher]
Key: Kallkey
Plaintext: BIT413B Stream Cipher Demo
Ciphertext (hex): b1b26817db06c2ba142d0e937d92b79ea3c8da50623a95172c5fc ...
Decrypted: BIT413B Stream Cipher Demo
Status: ✓

[Performance Test]
Encrypted 10,000 bytes in 5.22 ms
Speed: 1916.7 KB/s

[Stream Cipher Property]
Encryption: Ciphertext = Plaintext ^ Keystream
Decryption: Plaintext = Ciphertext ^ Keystream
Same operation for both (XOR is symmetric)

✓ LFSR implemented
✓ RC4 working correctly
✓ Performance tested

(crypto-env)-(darkwood@voidwalker)-[~/kali-crypto-toolkit]
```

Week 4: AES Block Cipher

Fig 4: AES-256-CBC Key Generation, Encryption/Decryption, Performance Test, and Security Features

```
Session Actions Edit View Help
(crypto-env)-(darkwood@voidwalker)-[~/kali-crypto-toolkit]
python3 ~/kali-crypto-toolkit/scripts/week4_aes_short.py

=====
WEEK 4: AES BLOCK CIPHER
=====

[AES Key Generation]
Algorithm: AES-256-CBC
Key size: 256 bits
Key (hex): 4014d22b4c3a3098e05aa87034f351bf38966f3 ...
Block size: 128 bits (16 bytes)
Mode: CBC (Cipher Block Chaining)

[Encryption & Decryption]
Plaintext: b'Confidential: Project launch date is 2026-12-01'
Ciphertext (hex): 2083c4d422ba6dffb00c1225f6f9ca0217af1d72d30be0dd5454b2e0abff6d2506851249a15e73 ...
Ciphertext size: 64 bytes
Encryption time: 2.61 ms
Decrypted: b'Confidential: Project launch date is 2026-12-01'
Decryption time: 0.09 ms
Status: ✓ PASS

[File Encryption Demo]
Original size: 47 bytes
Encrypted size: 64 bytes
Overhead: 17 bytes (IV + padding)

[Performance Test]
Encrypted 100 KB in 0.46 ms
Speed: 215.0 MB/s

[Security Features]
✓ Random IV prevents identical plaintext patterns
✓ CBC mode chains blocks for better security
✓ PKCS7 padding handles non-16-byte data
✓ AES-256 provides 256-bit security strength

=====
WEEK 4 COMPLETE - Ready for Week 5: RSA
=====

(crypto-env)-(darkwood@voidwalker)-[~/kali-crypto-toolkit]
```


Week 5: RSA Public Key Cryptography

Fig 5: RSA-2048 Key Pair Generation, Public Key Encryption, Private Key Decryption, and Security Comparison

```
(crypto-env)darkwood@voidwalker: ~/kali-crypto-toolkit
Session Actions Edit View Help
(crypto-env)~(darkwood@voidwalker)~/kali-crypto-toolkit
$ python3 ~/kali-crypto-toolkit/scripts/week5_rsa_short.py

=====
WEEK 5: RSA PUBLIC KEY CRYPTOGRAPHY
=====

[RSA Key Pair Generation]
Algorithm: RSA
Key size: 2048 bits
Public exponent: 65537
Generation time: 23.89 ms
Public key (first 60 chars): -----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQAM...

[Public Key Encryption]
Original: Secret meeting at 3pm in Lab 4B
Ciphertext (hex): 827d972cb13bf1b1489cdabe4f243c4e12620cc86dd95 ...
Ciphertext size: 256 bytes
Encryption time: 0.44 ms

[Private Key Decryption]
Decrypted: Secret meeting at 3pm in Lab 4B
Decryption time: 0.96 ms
Status: ✓ PASS

[Security & Comparison]
Asymmetric vs Symmetric:
- RSA uses two keys (public/private)
- AES uses one key (symmetric)
Max message size (2048-bit): 190 bytes with OAEP
Security level: ~112 bits (equivalent to AES-112)
Use case: Key exchange, digital signatures, small data

[Hybrid Encryption Note]
RSA is 100x slower than AES
Real systems: RSA for key exchange + AES for bulk data

=====
(crypto-env)~(darkwood@voidwalker)~/kali-crypto-toolkit
$
```

Week 6: Hashing and Password Security

Fig 6: SHA-256 Hash Generation, PBKDF2 Password Hashing with Salt, Login Authentication, and Security Analysis

```
(crypto-env)darkwood@voidwalker: ~/kali-crypto-toolkit
Session Actions Edit View Help
(crypto-env)~(darkwood@voidwalker)~/kali-crypto-toolkit
$ python3 ~/kali-crypto-toolkit/scripts/week6_hashing_short.py

=====
WEEK 6: HASHING AND PASSWORD SECURITY
=====

[SHA-256 Hash Function]
Message: B1f5138 Advanced Cryptography
SHA-256: dcb38ae2ab646f4bca7597fca3f96c8661a03fcd2d2cc2e3729eb39dd77fc5
Hash length: 64 chars (256 bits)
Property: One-way - cannot reverse to get original

[Password Hashing with Salt - PBKDF2]
Password: MySecurePassword123
Salt: 40d29e08e0b0bc2a4a00d2c4ac720b7e ...
Hash 1: 999b1ec0e0a1acd0148afdc0b2cbce0 ...
Salt 2: 17c288e0d109f13083081c1da20cf3a1 ...
Hash 2: 17488b5d8d177cce77e2acd6a0b5cd8 ...
Same password, different salts ✓
Reason: Different random salts produce different hashes

[Login Authentication Simulation]
Login 'correct_password123': ✓ ACCESS GRANTED
Login 'wrong_password': ✗ ACCESS DENIED

[Security Analysis]
Why hashing is irreversible:
- Cryptographic hash functions are one-way
- Small changes produce completely different hashes
- Avalanche effect: 1-bit change changes ~50% of hash bits
Importance of salt:
- Prevents rainbow table attacks
- Same password → different hash per user
PBKDF2 iterations (100,000):
- Slows down brute-force attacks
- Tunable for future hardware improvements

[Password Strength Test]
Weak password: password123
Hash: ef2b778baf7710826450b9ecbcb0a44ae106c ...
Status: Easily cracked - dictionary attack
Strong password: X9mL1$mq2/9u6q!
Hash: f4d5ccc7f13290120986a9a9efc433448b66ab ...
Status: Secure - High entropy

=====
(crypto-env)~(darkwood@voidwalker)~/kali-crypto-toolkit
$
```

Creativity

```
(crypto-env)darkwood@voidwalker: ~/kali-crypto-toolkit/creative
$ ls
01_cover_banner.txt 02_crypto_timeline.txt 03_algorithm_comparison.html 04_crypto_cheatsheet.txt 05_certificate.txt 06_python_banner.py 07_creative_summary.txt create_all_creative.sh
(crypto-env)darkwood@voidwalker: ~/kali-crypto-toolkit/creative
$ cat ~/kali-crypto-toolkit/creative/05_certificate.txt

-----
CERTIFICATE OF COMPLETION
BIT4138: Advanced Cryptography

Presented to: Gakere Samuel
Admission: BSCCS/2024/55278
Date: 2026

Completed Weeks 1-6

[X] Environment Setup [X] Classical Ciphers [X] Stream Ciphers
[X] AES Block Cipher [X] RSA Public Key [X] Hashing & PWD

-----
SIGNED
Lecturer: MR NYORO MICHAEL
Date: 2026
-----

(crypto-env)darkwood@voidwalker: ~/kali-crypto-toolkit/creative
$ sudo npm install @express-prisma/prisma/client bcrypt jsonwebtoken multer dotenv
```

Week 7: Digital Signatures & Certificates

Fig 7: RSA Digital Signature Generation, Verification, Tamper Detection, and X.509 Certificate Structure

```
(crypto-env)darkwood@voidwalker: ~/kali-crypto-toolkit
$ ls
creative crypto-env outputs README.md Report.odt Report.pdf Reports.odg requirements.txt screenshots scripts txt.py typescript
(crypto-env)darkwood@voidwalker: ~/kali-crypto-toolkit
$ python3 ~/kali-crypto-toolkit/scripts/week7_signatures_short.py

WEEK 7: DIGITAL SIGNATURES AND CERTIFICATES
BIT4138 - Advanced Cryptography
Student: Gakere Samuel
Lecturer: MR NYORO MICHAEL

[1] DIGITAL SIGNATURE KEY PAIR GENERATION
Algorithm: RSA-PSS with SHA-256
Key size: 2048 bits
Generation time: 87.70 ms
Purpose: Authenticity + Integrity (not confidentiality)

[2] DIGITAL SIGNATURE GENERATION
Document: Official Contract: Payment of KES 500,000 to CryptoLab
Signature (hex): 1a2e77b7afe1be39d1e43bfcf79a7d5c7b1a7952994c892d346115daef3 ...
Signature size: 256 bytes
Signing time: 1.95 ms

[3] SIGNATURE VERIFICATION
Result: VALID SIGNATURE
Verification time: 0.12 ms
Document integrity: INTACT
Signer identity: CONFIRMED

[4] TAMPERED DOCUMENT DETECTION
Original document with tampering detected
Result: SUCCESS - Tampered document rejected
Reason: InvalidSignature - Signature verification failed

[5] X.509 CERTIFICATE STRUCTURE
-----
X.509 CERTIFICATE EXAMPLE
-----
| Version: 3
| Serial Number: 55:66:77:88
| Signature Algorithm: sha256WithRSAEncryption
| Issuer: CN=KaliCA, O=Kali Linux, C=US
| Validity:
|   Not Before: 2026-06-01 00:00:00 GMT
|   Not After: 2027-06-01 00:00:00 GMT
| Subject: CN=Gakere Samuel, O=University, C=KE
| Public Key: RSA (2048 bits)
| X509v3 Extensions:
|   - Key Usage: Digital Signature, Key Encipherment
|   - Extended Key Usage: TLS Client Authentication
|-----

[6] SECURITY PROPERTIES
Digital Signatures provide:
- AUTHENTICITY: Confirms signer's identity
- INTEGRITY: Detects any document tampering
- NON-REPUTATION: Signer cannot deny signing
```

```
Real-world applications:
- Software distribution (code signing)
- Legal documents (digital contracts)
- Email security (S/MIME, PGP)
- TLS/SSL certificates
```

```
(crypto-env)-(darkwood@voidwalker)-[~/kali-crypto-toolkit]
```

Reflection:

Digital signatures use RSA private key to sign documents (Fig 7). Signing time and verification are fast. Any tampering invalidates the signature demonstrated by rejected modified document. X.509 certificates bind identity to public keys. Provides authenticity, integrity, and non-repudiation for legal documents and software distribution.

Week 8: PKI, Digital Certificates & Diffie-Hellman

Fig 8: Diffie-Hellman Key Exchange, PKI Components, X.509 Certificate Structure, SSL/TLS Handshake, and Symmetric vs Asymmetric Comparison

```
Session Actions Edit View Help
(crypto-env)-(darkwood@voidwalker)-[~/kali-crypto-toolkit]
$ python3 ~/kali-crypto-toolkit/scripts/week0_pki.py

WEEK 8: PKI, DIGITAL CERTIFICATES & DIFFIE-HELLMAN
BIT4138 - Advanced Cryptography
Student: Gakere Samuel
Lecturer: MR NYORO MICHAEL

[1] DIFFIE-HELLMAN KEY EXCHANGE DEMONSTRATION

Public Parameters:
Prime (p): 23
Generator (g): 5

Private Keys (Secret):
Alice's private key: 6
Bob's private key: 15

Public Keys (Exchanged):
Alice sends: 8
Bob sends: 19

Shared Secret:
Alice computes: 2
Bob computes: 2
Status: ✓ MATCH - Secure key exchange successful

[2] X.509 CERTIFICATE STRUCTURE

+-----+
| X.509 DIGITAL CERTIFICATE |
+-----+
| Version: 3 |
| Serial Number: 12:34:56:78:9A:BC |
| Signature Algorithm: sha256withRSAEncryption |
| Issuer: CN=DigiCert, O=DigiCert Inc, C=US |
| Validity: |
|   Not Before: 2026-01-01 00:00:00 GMT |
|   Not After: 2027-01-01 23:59:59 GMT |
| Subject: CN=example.com, O=Example Corp, C=KE |
| Public Key Algorithm: RSA (2048 bits) |
| Public Key: [hexadecimal representation] |
| X509v3 Extensions: |
|   - Key Usage: Digital Signature, Key Encipher |
|   - Extended Key Usage: TLS Server Auth |
|   - Subject Alternative Name: DNS:example.com |
| Signature: [CA's digital signature] |
+-----+

[3] PKI COMPONENTS AND THEIR ROLES

Certificate Authority (CA):
Trusted entity that issues and revokes certificates
Registration Authority (RA):
Verifies user identity and approves certificate requests
Digital Certificate:
Binds public key to an identity
Certificate Repository:
Stores and provides access to certificates
End User:
```

```
Session Actions Edit View Help
End User:
Individual or system using certificates

[4] CERTIFICATE LIFECYCLE


| Step | Phase                             | Description                                  |
|------|-----------------------------------|----------------------------------------------|
| 1    | Key Pair Generation               | User generates public/private key pair       |
| 2    | Certificate Signing Request (CSR) | User sends public key and identity to CA     |
| 3    | Identity Verification             | CA verifies user identity                    |
| 4    | Certificate Issuance              | CA signs and issues certificate              |
| 5    | Certificate Usage                 | Certificate used for HTTPS, email, etc.      |
| 6    | Renewal or Revocation             | Certificate renewed before expiry or revoked |



[5] CERTIFICATE REVOCATION METHODS
CRL (Certificate Revocation List):
- List of revoked certificates published by CA
- Periodically downloaded by clients
- Can be delayed
OCSP (Online Certificate Status Protocol):
- Real-time certificate status checking
- Query sent to OCSP responder
- Immediate response

[6] SSL/TLS HANDSHAKE WITH PKI


| Phase               | Description                                      |
|---------------------|--------------------------------------------------|
| Client Hello        | Client sends supported ciphers and TLS version   |
| Server Hello        | Server selects cipher and sends random value     |
| Certificate         | Server sends digital certificate with public key |
| Server Key Exchange | DH or ECDHE parameters (if needed)               |
| Certificate Request | Server requests client certificate (optional)    |
| Server Hello Done   | Server indicates end of handshake messages       |
| Client Key Exchange | Client sends premaster secret                    |
| Change Cipher Spec  | Both parties switch to negotiated cipher         |
| Finished            | Both parties verify handshake integrity          |



[7] SYMMETRIC VS ASYMMETRIC COMPARISON


| Feature          | Symmetric       | Asymmetric         |
|------------------|-----------------|--------------------|
| Number of Keys   | 1 (shared)      | 2 (public/private) |
| Speed            | Fast            | Slow               |
| Key Distribution | Difficult       | Easy               |
| Scalability      | Low             | High               |
| Examples         | AES, DES, 3DES  | RSA, DH, ECC       |
| Use Case         | Bulk encryption | Key exchange, Sign |



[8] DISCRETE LOGARITHM PROBLEM EXPLANATION
The security of Diffie-Hellman relies on the Discrete Logarithm Problem:
Given:
p = prime number (public)
g = generator (public)
A = g^a mod p (public)
```

Reflection:

Diffie-Hellman enables secure key exchange without pre-shared secrets (Fig 8). PKI provides trust through Certificate Authorities issuing X.509 certificates. SSL/TLS handshake uses certificates for authentication. Discrete logarithm problem makes DH computationally secure. Symmetric encryption is faster for bulk data while asymmetric handles key exchange.

Week 9: RSA Cryptosystem, Miller-Rabin Primality Test & Pollard Rho

Fig 9-10: Miller-Rabin Primality Test, RSA Key Generation & Encryption, Digital Signatures, ElGamal Cryptosystem, and RSA vs ElGamal vs ECC Comparison

```
(crypto-env)darkwood@voidwalker: ~/kali-crypto-toolkit
Session Actions Edit View Help
(crypto-env)~(darkwood@voidwalker)~(~/kali-crypto-toolkit)
$ python3 ~/kali-crypto-toolkit/scripts/week9_rsa_millerrabin.py
WEEK 9: RSA CRYPTOSYSTEM, MILLER-RABIN & POLLARD RHO

[1] MILLER-RABIN PRIMALITY TEST
Testing primality of numbers:
Number | Result
-----|-----
2 | Prime
3 | Prime
4 | Composite
5 | Prime
7 | Prime
11 | Prime
13 | Prime
17 | Prime
19 | Prime
23 | Prime

Testing 10 random numbers:
Number | Result
-----|-----
113 | Prime
559 | Composite
613 | Prime
340 | Composite
577 | Prime
801 | Composite
255 | Composite
963 | Composite
869 | Composite
829 | Prime

[2] RSA KEY GENERATION
Prime p: 61
Prime q: 53
Modulus n: 3233
Totient phi: 3120
Public exponent e: 17
Private exponent d: 2753

Public Key: (e=17, n=3233)
Private Key: (d=2753, n=3233)

[3] RSA ENCRYPTION & DECRYPTION
Original Message: HELLO
ASCII Values: [72, 69, 76, 76, 79]
Encrypted (ciphertext): [3880, 28, 2726, 2726, 1307]
Decrypted ASCII: [72, 69, 76, 76, 79]
Decrypted Message: HELLO
Status: SUCCESS

[4] RSA DIGITAL SIGNATURE
Message: HELLO
SHA-256 Hash: 3733cd977ff8eb18b987357e22ced99f46097f31 ...

Status: SUCCESS

[4] RSA DIGITAL SIGNATURE
Message: HELLO
SHA-256 Hash: 3733cd977ff8eb18b987357e22ced99f46097f31 ...
Digital Signature: 2793
Recovered Hash: 502
Original Hash: 926141847
Signature Status: INVALID

[5] POLLARD RHO FACTORIZATION
Factoring n = 8051
Found factor: 97
Other factor: 83

WEEK 9 COMPLETE

(crypto-env)~(darkwood@voidwalker)~(~/kali-crypto-toolkit)
```

Reflection:

Miller-Rabin efficiently tests primality for RSA key generation. RSA uses $p=61$, $q=53$ with $e=17$ producing working encryption/decryption. Digital signatures provide authentication using SHA-256 hashing. Pollard Rho factorization demonstrates vulnerability of small prime numbers. Security depends on difficulty of factoring large integers.

Week 10: ElGamal Cryptosystem & ECC

Fig 10: ElGamal Key Generation, Encryption, Decryption, Randomness Demonstration, RSA vs ElGamal Benchmark, and ECC Concepts

```
Session Actions Edit View Help
(crypto-env)-(darkwood@voidwalker)-[~/kali-crypto-toolkit]
$ python3 ~/kali-crypto-toolkit/scripts/week10_elgamal_ecc.py

WEEK 10: ELGAMAL CRYPTOSYSTEM & ECC

[1] ELGAMAL KEY GENERATION
-----
Prime p: 23
Generator g: 5
Private Key x: 6
Public Key y: 8

[2] ELGAMAL ENCRYPTION
-----
Plaintext M: 10
Random k: 7
Ciphertext C1: 17
Ciphertext C2: 5

[3] ELGAMAL DECRYPTION
-----
Recovered Plaintext: 10
Status: SUCCESS

[4] ELGAMAL RANDOMNESS DEMONSTRATION
-----
Original Message: 15
Encryption with different random k values:
k | C1 | C2
--|---|---
3 | 10 | 21
7 | 17 | 19
11 | 22 | 15
15 | 19 | 7
19 | 7 | 14

Same plaintext produces different ciphertexts

[5] RSA vs ELGAMAL BENCHMARK
-----
ElGamal (100 operations): 0.0539 ms
RSA (100 operations): 0.0553 ms
Note: RSA is faster for these small numbers

[6] ELLIPTIC CURVE CRYPTOGRAPHY (ECC)
-----
Elliptic Curve Equation:  $y^2 = x^3 + ax + b$ 
Example:  $y^2 = x^3 - 3x + 5$ 

ECC Point Operations:
- Point Addition:  $P + Q = R$ 
- Point Doubling:  $2P = R$ 
- Scalar Multiplication:  $kP = R$ 

ECC Advantages:
- 256-bit ECC  $\approx$  3072-bit RSA security
- Faster computations
- Lower power consumption
- Suitable for IoT and mobile devices

Applications for IoT and mobile devices:
- Mobile banking
- Smart cards
- Cryptocurrencies
- Secure messaging

[7] ELGAMAL APPLICATIONS
-----
ElGamal is used in:
- Digital signatures (ElGamal signature scheme)
- Diffie-Hellman key exchange
- Encryption systems
- Secure email
- VPN authentication

Security Note:
- Security relies on Discrete Logarithm Problem
- Random k must be unique for each encryption
- Reusing k compromises security

WEEK 10 COMPLETE

(crypto-env)-(darkwood@voidwalker)-[~/kali-crypto-toolkit]
$
```

Reflection:

ElGamal uses discrete logarithm problem with randomized encryption - same plaintext produces different ciphertexts each time. Random k values (3,7,11,15,19) demonstrate this property. ECC offers smaller keys (256-bit $\approx$ 3072-bit RSA) making it ideal for mobile and IoT devices. Used in banking, cryptocurrencies, and secure messaging.